PROJECT CASE STUDY

Trulle Portal

A cinematic WebGL navigation system that turns a website's line-art into an interactive mask — hiding destinations behind hover-triggered "peepholes" instead of a conventional menu.

SYSTEM	SCOPE	STATUS
Frontend Motion & Rendering System	**Frontend · WebGL · Animation · WordPress Plugin**	**Packaged & Deployed as a Client WordPress Plugin**

01 The Problem

Standard website navigation — menus, nav bars, button grids — treats the destination as a functional afterthought bolted onto whatever the page looks like. For a client whose brand identity centers on a single continuous piece of custom line-art, that’s a real conflict: a nav bar breaks the artwork’s continuity, and plain clickable hotspots feel disconnected from the illustration they sit on.

Trulle needed navigation that treats the artwork itself as the interface, not something competing with it — and needed it shipped into a real non-developer toolchain (Elementor), not a one-off developer demo.

> Trulle Portal was built to make the artwork itself the entire navigation system — five portals hidden inside one continuous line drawing, revealed only on interaction, with no visible menu, button, or link anywhere on the page.

02 What Was Built

Trulle Portal is built as four coordinated pieces — a rendering system, a motion system, a state architecture, and a client-facing configuration layer — each independent, then wired together into one shippable widget.

CINEMATIC PEEPHOLE REVEAL

A full-bleed 4K SVG mask sits over hidden preview media (image, video, or embedded page). Hovering a portal opens a ring-shaped hole to reveal it; clicking expands that hole until it covers the viewport and cross-fades into the real destination.

WEBGL RENDERING LAYER

A PixiJS 8 application renders every ring, mask, and preview sprite as GPU-accelerated graphics, synced frame-for-frame to a single GSAP ticker instead of running two competing render loops.

MULTI-INSTANCE STATE ARCHITECTURE

A per-instance Zustand store, delivered through React context rather than a module singleton, so multiple independent portal widgets can share one page without ever clobbering each other's state.

ELEMENTOR CONFIGURATION WIDGET

A packaged WordPress plugin exposing a repeater-field GUI — destination URL, preview media, label, enabled toggle — so non-developer editors can configure all five portals without touching JSON or code.

03 How It Works

The frontend is built as three synchronized layers, each with a single responsibility — this separation means the WebGL rendering code never owns interaction logic, and the interaction layer never touches rendering internals directly.

Layered rendering

The static SVG art layer is purely visual. The PixiJS layer owns every dynamic Graphics/Sprite object per portal and stashes them into the shared store so the animation layer can reach into WebGL objects directly. The hit-box layer is a set of invisible, absolutely-positioned overlays that own all user interaction.

Motion system

Each portal gets one cached, paused GSAP timeline with three labels — rest → hover → open. Every interaction just tweens or seeks along that existing timeline. Ring growth is a radius redraw of the Graphics object at each tweened frame, not a scale transform, so the stroke stays crisp at any size.

Destination loading

A single destination iframe is reused across all portals and de-duplicates preload/open requests by resolved target key. Image and video destinations load via a generated srcdoc document; plain URLs load directly via iframe.src.

Forward & back navigation

Clicking tweens the timeline to open, cross-fades in the destination once the ring covers the viewport, and pushes browser history. On popstate, Pixi state is seeked instantly to open (no flash to rest) before tweening back down, recovered via a short-lived session cache.

04 Key Features

- ✓ **Peephole hover reveal —** hovering a portal opens a masked hole showing hidden preview media, with no visible hotspot or outline at rest.
- ✓ **Seamless click-to-destination expansion —** the ring grows past the viewport bounds and cross-fades into the real page, with zero navigation flicker.
- ✓ **Multi-portal, multi-instance support —** up to five configurable portals per widget, and multiple widget instances per page, each fully isolated.
- ✓ **Configurable per-portal content —** destination, preview media, label text, and text color are all runtime-injected, never hardcoded.
- ✓ **Image, video, and URL preview support —** media previews render inline via a generated document; plain destinations load as a live embedded page.
- ✓ **Crisp-at-any-size ring rendering —** geometry is redrawn at each animation frame instead of scaled, so stroke width never distorts.
- ✓ **Preload de-duplication —** repeated hovers on the same portal never re-trigger a redundant destination fetch.
- ✓ **Instant, flash-free back navigation —** browser back restores the correct portal state immediately, even after a hard reload.
- ✓ **No-code Elementor configuration —** a GUI widget with repeater fields replaces hand-written JSON for every portal's content.

05 Technical Approach

For technical readers: the system was built with production reliability and clean separation of concerns as first-order priorities, not afterthoughts.

LAYER	STACK	PURPOSE
Rendering	PixiJS 8 (WebGL)	GPU-accelerated ring / mask / preview graphics
Animation	GSAP + @gsap/react	Single cached timeline per portal; drives Pixi redraws and iframe cross-fades
UI / State	React 19, Zustand 5	Per-instance store via context; interactive hit-box overlays
Build	Vite 6	Dependency-free IIFE library bundle + CSS
Integration	PHP (WordPress / Elementor)	Repeater-field GUI emitting normalized data-portals JSON

Design decisions that matter

- ✓ **Radius redraw, not scale transforms.** Ring growth redraws the Graphics geometry every tweened frame instead of scaling a sprite, so the stroke stays exactly as crisp at full-viewport size as it does at rest.
- ✓ **Per-instance state, not a singleton.** Every portal system gets its own Zustand store through React context, so the same widget can be mounted multiple times on one page — a direct requirement of shipping into Elementor.
- ✓ **One render loop, not two.** Pixi's own ticker is stopped entirely; a single GSAP ticker drives every Pixi redraw, eliminating the risk of two competing animation loops drifting out of sync.
- ✓ **Config precedence, not hardcoding.** Content flows in through JSON overrides, per-attribute overrides, then Elementor-mapped nodes, merged by precedence per field — so geometry and content never live in the same place.

06 Value Delivered

This is the difference between a decorative site and a designed interaction:

Cinematic	Frame-Perfect	Editor-Ready
NOT GENERIC	NOT APPROXIMATE	NOT DEVELOPER-LOCKED

- ✓ **Brand-first navigation.** The artwork stops competing with the menu — it is the menu, with no bar, button, or link breaking its continuity.
- ✓ **No visible seams.** WebGL-rendered rings and a de-duplicated destination frame mean hover, click, and page transition all happen without a flicker, flash, or loading state.
- ✓ **Scales to real content teams.** The Elementor widget lets non-developers configure every portal's destination, media, and copy — no code changes needed after handoff.
- ✓ **Safe to extend.** New portals, preview types, or destinations are configuration changes, not new code — the rendering and motion systems are content-agnostic by design.
- ✓ **Robust to real browser behavior.** Back-navigation, hard reloads, and repeated hovers are all handled explicitly, rather than left to accidental default behavior.

Trulle Portal — a WebGL-driven cinematic navigation system that replaces a conventional menu with an interactive artwork mask, packaged as a production-ready WordPress/Elementor plugin.